

AI Test Data Generator — Utility Documentation

Author: Solomon Eshiet | Part 6B.2 | Date: 3 August 2026

1. What the Utility Does

The script at `ai/test-data-generator.ts` solves a practical QA problem: edge-case test data for transfer-limit features is slow to hand-author and error-prone at boundary values. It accepts a scenario type (`boundary` , `concurrent` , `schedule` , or `reversal`), calls an LLM with the Digital Wallet Transfer Limits spec, validates the structured JSON response against known tier caps, and writes the result to `ai/generated-test-data.json` . It accelerates parametrized test seeding; it does not replace human oracle review.

2. Prompt Design Rationale

Prompt used (boundary mode):

```
You are a QA test-data generator. Use this product spec:
Digital Wallet Transfer Limits:
- Tier 1: max 20,000 per transaction, 50,000 per day.
- Tier 2: max 200,000 per transaction, 500,000 per day.
- Tier 3: no per-transaction limit, 5,000,000 per day.
[...remaining spec rules...]

Return ONLY valid JSON (no markdown fences, no prose) matching this shape:
{ "scenarioType": "boundary", "scenarios": [ ... ] }
Use realistic edge-case values: amounts at/just under/just over limits...
Generate exactly 6 boundary scenarios (2 per tier)...
Include at least one own_linked_bank exempt case above Tier 1 caps.
```

Spec as context anchors the model to product rules. Without it, the LLM invents limits (e.g. 25,000 Tier 1 caps) and omits exemptions.

JSON not prose enables parsing, schema validation, and fixture import. Prose output requires manual transcription and introduces copy errors.

Explicit tier limits in the prompt align generation with the validator's `TIER_LIMITS` constants. Without repetition, prompt and validation drift apart.

Scenario-specific schemas (concurrent timing arrays, schedule dates) constrain each CLI mode. A generic prompt produces incomplete or irrelevant fields.

3. Example Input and Output

```
npm run generate:data -- boundary
```

```
{
  "scenarioType": "boundary",
```

```
"scenarios": [
  { "id": "BOUND-T1-02", "tier": 1, "transferAmount": 20001, "dailySpentBefore":
0, "expectedOutcome": "reject" },
  { "id": "BOUND-T1-03", "tier": 1, "transferAmount": 2000, "dailySpentBefore":
49000, "expectedOutcome": "reject" },
  { "id": "BOUND-T1-EXEMPT", "tier": 1, "transferAmount": 100000,
"dailySpentBefore": 50000, "recipientType": "own_linked_bank", "expectedOutcome":
"accept" }
],
"validationWarnings": []
}
```

- **20001** — just-over Tier 1 per-transaction boundary (20,000 max).
- **49000 + 2000** — daily limit exhaustion with 1,000 remaining.
- **100000 to own_linked_bank** — Rule 4 exemption when daily wallet limit is already exhausted.

4. Validation Approach

Checks run before write: JSON parse; `scenarioType` and non-empty `scenarios`; required fields per mode; $\text{tier} \in \{1,2,3\}$; positive integer amounts; cross-check against tier caps — flag `accept` when amount exceeds per-transaction or daily limit (except `own_linked_bank`); flag unrealistic preconditions. Warnings go to `validationWarnings`; structural failures abort.

Failing example: Tier 1, `transferAmount: 25000`, `expectedOutcome: "accept"`, `recipientType: "wallet"` → warning: exceeds 20,000 per-transaction limit — the same class of hallucination identified in Part 6A critique.

5. Limitations and When Not to Use This

(1) Cannot resolve spec ambiguities (midnight timezone, schedule booking vs execution) — may emit plausible but unconfirmed oracles. **(2)** Rule-based validation misses semantic errors (wrong concurrent race outcome with valid JSON).

A human tester beats this utility when scoping risk, marking tests blocked pending product answers, and signing off financial-control oracles. Use it to draft data; not to approve limit-enforcement correctness.

End of document